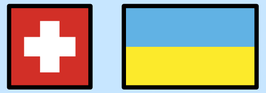
ISO/IEC 19510:2015
SNIA KV 2020



Scope: **ERP/1: BPMN Process Engine**

ERP/1: KV Storage

https://erp.uno/man/ua/1.htm — ERP/1 Handbook (Ukrainian)

https://erp.uno/book/erp.pdf — Electronic Governmental Systems

https://cafe.groupoid.space — Groupoid Infinity Cafe

https://zencrypted.uk — Zen Crypted International

Theory

BPMN

- Basic XML Format Support
- 1:1 (BPMN:ERLANG) Correspondence
- Documents Context Support
- KVS Backends
- Pub Sub Integration
- IAM/ABAC Integration

KVS

- Direct B-Tree (LSM) retrieval
- Natural Memory Caching
- Optimized for Online Streaming
- Optimized for SSD SNIA KVS API 1.1
- RocksDB compatibility
- MNESIA compatibility
- HRL Schema

Practice

EXO and FIN

- FORM/NITRO/N2O
- KVS/BPE/SYN
- ADM

SAMPLE

- NITRO
- KVS
- SYN
- N2O

ERP/1: BPMN Processes SYNRC BPE

https://n2o.dev/books/bpe.pdf

Erlang Company Since 2013

https://n2o.dev/articles/history.htm

https://bpe.n2o.dev/

Microsoft Oracle Red Hat

https://erp.uno/bpmn/

SYNRC BPE

ISO/IEC 19510:2015

ERP/1: Processes

Governmental and Corporate Deployments

Worldwide:

- Biggest Eastern European Bank ~30M subscribers (Deposit Application)
- Ministry of Internal Affairs (Including National Guard Forces)
- National Weapon Registry of MIA Ukraine
- NYNJA Communicator

BPE/BPMN Practical Model (System F/MLTT)

```
axiom start : Proc -> IO Sup
axiom stop : String -> IO Sup
axiom next : ProcId -> IO ProcRes
axiom amend : Π (k: U), ProcId -> k -> IO ProcRes
axiom discard : Π (k: U), ProcId -> k -> IO ProcRes
axiom modify : Π (k: U), ProcId -> k -> Atom -> IO ProcRes
axiom event : ProcId -> String -> IO ProcRes
axiom head : ProcId -> IO Hist
axiom hist : ProcId -> IO (List Hist)
```

Zen Crypted

BPE/BPMN Morphisms (Functions)

API

- LOAD (Load Process State)
- START (Start Process under Supervision)
- PROC (Process Cache)
- NEXT (Stage Completion with Optional Target State Specify)
- COMPLETE (Complete the Stage of the Process)
- AMEND (Modification of Process State with Process Tick)
- DISCARD (Modification of Process State with Process Tick)
- APPEND (Modification of Process State)
- REMOVE (Modification of Process State)
- EVENT (Trigger Event)
- HIST (Get Process Trace)
- TASK (Get Task Info)
- DOC (Search Documents in Context)

Zen Crypted

BPE/BPMN Objects (Schema)

#process
#monitor
#continue
#lock

#task
#beginTask
#endTask
#userTask
#serviceTask
#receiveTask
#sendTask
#gateway

#sequenceFlow

#event
#boundaryEvent
#timeoutEvent
#messageEvent

#history

Process Orchestration Prerequisites

Overview

Process Management is needed if these criteria meet at scale:

- Formal Definition of the Process
- Process Governance (Versioning, Archiving)
- Manual Edition
- Human Code Review of the Process
- Process Audit (Public Key Infrastructure)
- Process Automation
- Process Tracing
- Governmental and Defense Contracts

History of Processing Languages

— EMAIL: FSM
— Event-Condition-Action Reactive Rule Engines
— Expert Systems: RETE Engine, Prolog
— Workflow Standards of the past: XPDL, BPML, OpenWFE, WWF and jBPM
— Workflow Standard After 2008: BPMN
— Trading: TpML, Fix, business contract routers, cross-exchanges, arbitrage
— Business Contacts Virtual Machines: EVM, Script VM, aebytecode
— Business Contract Languages: Sophia, Solidity, Plutus
— MLTT Erlang Frameworks: Dhall, Morte, Henk, Per, Frank, Christine
— Interaction Networks Evaluators: Formality, Moonad
— Stream Processing: Oz, Erlang, np/ling, Futhark

Isomorphic Process Models

Overview

Process Management has a series of manifestation, most notable among them:

- BPMN
- Finite State Machines (gen_server, gen_fsm, gen_statem, etc.)
- SADT
- np/ling
- Event Condition Action
- Petri Nets
- Pi-Calculus
- Linear Types (Lock Calculus)
- Tensor Calculus (Symmetric Monoidal Categories)

Two Computational Models

Lambda Calculus (Alonzo Church)

Non-typed core of **Cartesian Closed Categories** inner language. Arguments of functions are **Scalars**.
Has theoretical flaw in its core evaluator due to single core nature of thinking.

Pi Calculus (Yves Lafont)

Non-typed core of **Symmetric Monoidal Categories** inner language. Arguments of functions are **Tensors** with forward only time axis access. Notable Applications: Parallel Concurrent (Modal) Models like Erlang, GPU parallelization (Maya Victor).

BPE/BPMN Bank Account Example

Definition 1. Bank Account

```
def() ->
  P =  #process { name = "IBAN Account",
        module = ?MODULE,
        flows = [
            #sequenceFlow { id="->Init", source="Created", target="Init"},
            #sequenceFlow { id="->Upload", source="Init", target="Upload"},
            #sequenceFlow { id="->Payment", source="Upload", target="Payment"},
            #sequenceFlow { id="Payment->Signatory", source="Payment", target="Signatory"},
            #sequenceFlow { id="Payment->Process", source="Payment", target="Process"},
            #sequenceFlow { id="Process-loop", source="Process", target="Process"},
            #sequenceFlow { id="Process->Final", source="Process", target="Final"},
            #sequenceFlow { id="Signatory->Process", source="Signatory", target="Process"},
            #sequenceFlow { id="Signatory->Final", source="Signatory", target="Final"} ],
        tasks = [
            #beginEvent { id="Created" },
            #userTask { id="Init" },
            #userTask { id="Upload" },
            #userTask { id="Signatory" },
            #serviceTask { id="Payment" },
            #serviceTask { id="Process" },
            #endEvent { id="Final" } ],
        beginEvent = "Created",
        endEvent = "Final",
        events = [ #messageEvent{id="PaymentReceived"},
                   #boundaryEvent{id='*', timeout=#timeout{spec={0, {10, 0, 10}}}} ] },

   P#process{tasks = bpe_xml:fillInOut(P#process.tasks,P#process.flows)}.
```

Zen Crypted

BPE/BPMN Support Ticket Example

Definition 2. Support Ticket

```
def() ->
    P = #process{
      name = "Support Ticket",
      module = ?MODULE,
      flows = [
        #sequenceFlow{id = "->Assign", source = "Create", target = "Assign"},
        #sequenceFlow{id = "Assign->Work", source = "Assign", target = "Work"},
        #sequenceFlow{id = "Work->Resolve", source = "Work", target = "Resolve"},
        #sequenceFlow{id = "Resolve->Closed", source = "Resolve", target = "Closed"},
        #sequenceFlow{id = "Escalate->Work", source = "Escalate", target = "Work"}
      ],
      tasks = [
        #beginEvent{id = "Create"},
        #userTask{id = "Assign"},
        #userTask{id = "Work"},
        #serviceTask{id = "Escalate"},
        #userTask{id = "Resolve"},
        #endEvent{id = "Closed"}
      ],
      beginEvent = "Create",
      endEvent = "Closed",
      events = [
        #messageEvent{id = "TicketUpdate"},
        #boundaryEvent{id = '*', timeout = #timeout{spec = {0, {24, 0, 0}}}}
      ]
    },
    P#process{tasks = bpe_xml:fillInOut(P#process.tasks, P#process.flows)}.
```

Zen Crypted

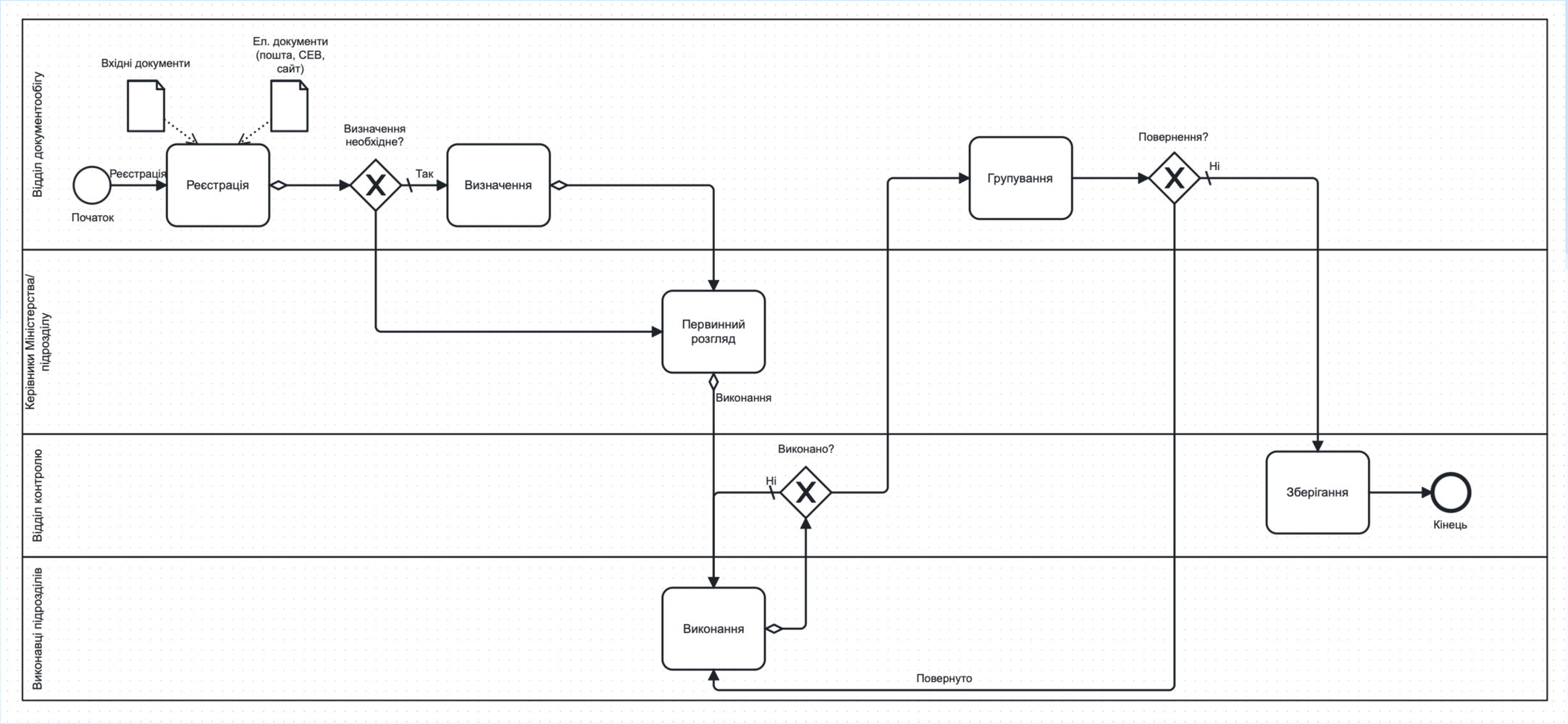
BPE/BPMN Purchase Order Example

Definition 3. Purchase Order

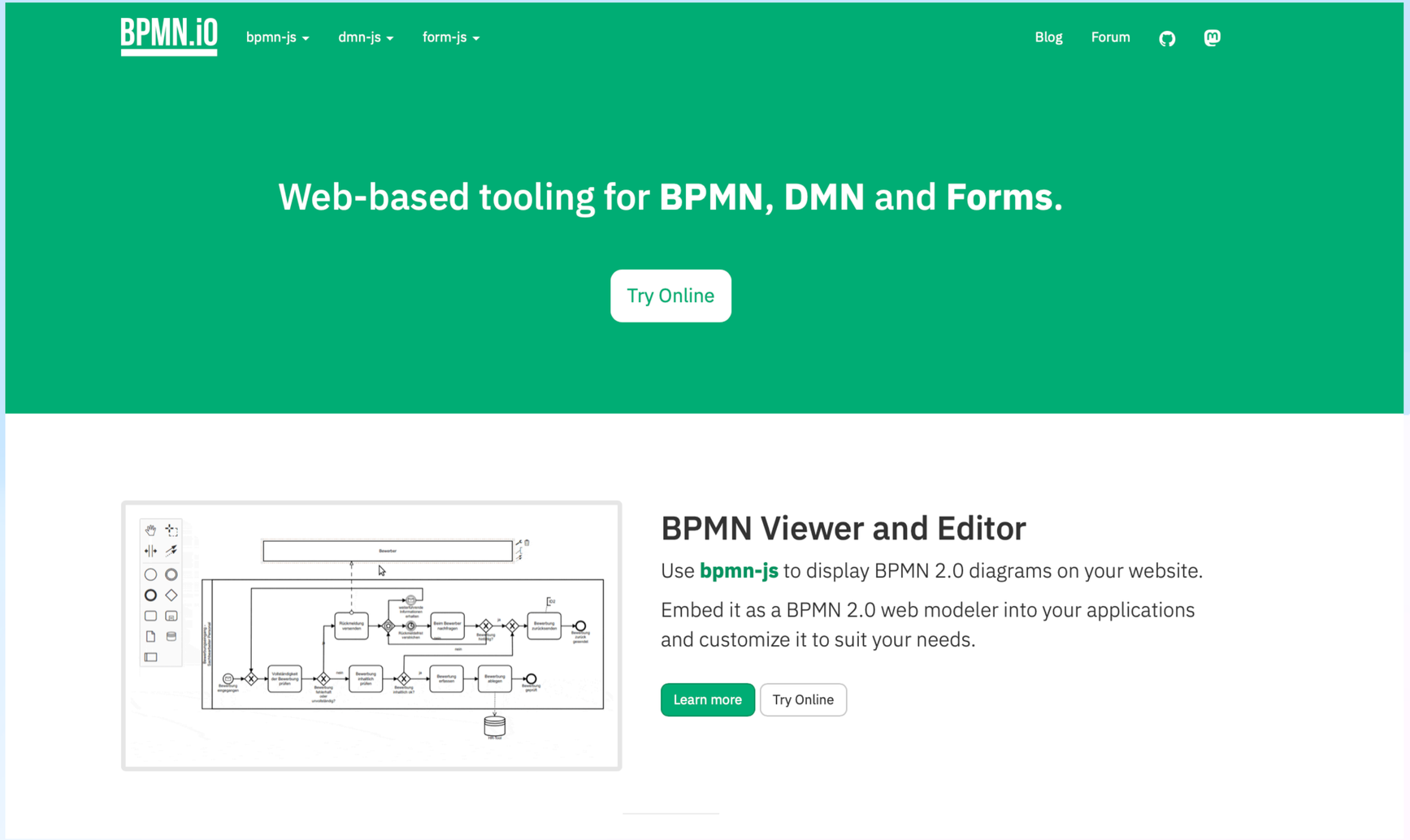
```
def() ->
    P = #process{
      name = "E-Commerce Order",
      module = ?MODULE,
      flows = [
        #sequenceFlow{id = "->CartActive", source = "CartCreated", target = "CartActive"},
        #sequenceFlow{id = "CartActive->Checkout", source = "CartActive", target = "Checkout"},
        #sequenceFlow{id = "AbandonedRecovery->CartActive", source = "AbandonedRecovery", target = "CartActive"},
        #sequenceFlow{id = "Checkout->Payment", source = "Checkout", target = "Payment"},
        #sequenceFlow{id = "Payment->Fulfilment", source = "Payment", target = "Fulfilment"},
        #sequenceFlow{id = "Fulfilment->PostPurchase", source = "Fulfilment", target = "PostPurchase"},
        #sequenceFlow{id = "PostPurchase->Delivered", source = "PostPurchase", target = "Delivered"}
      ],
      tasks = [
        #beginEvent{id = "CartCreated"},
        #userTask{id = "CartActive"},
        #serviceTask{id = "AbandonedRecovery"},
        #userTask{id = "Checkout"},
        #serviceTask{id = "Payment"},
        #serviceTask{id = "Fulfilment"},
        #serviceTask{id = "PostPurchase"},
        #endEvent{id = "Delivered"}
      ],
      beginEvent = "CartCreated",
      endEvent = "Delivered",
      events = [
        #messageEvent{id = "PaymentReceived"},
        #messageEvent{id = "ShipmentUpdate"},
        #boundaryEvent{id = '*', timeout = #timeout{spec = {0, {1, 0, 0}}}}
      ]
    },
    P#process{tasks = bpe_xml:fillInOut(P#process.tasks, P#process.flows)}.
```

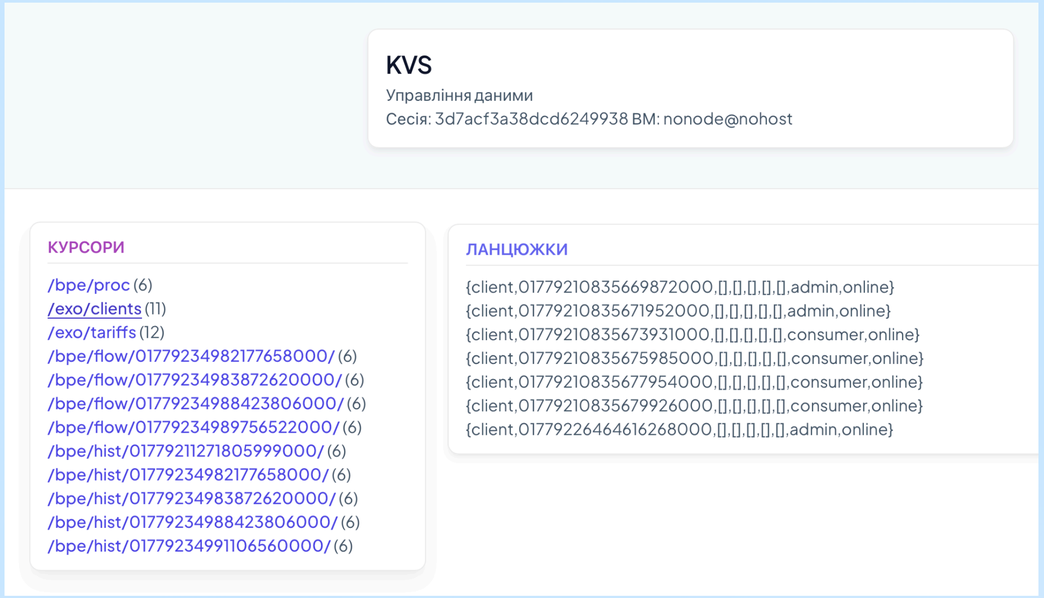
Zen Crypted

Real Process Example



bpmn.io


BPMN.iO
bpmn-js
dmn-js
form-js
Blog
Forum
Web-based tooling for BPMN, DMN and Forms.
Try Online
BPMN Viewer and Editor
Use bpmn-js to display BPMN 2.0 diagrams on your website.
Embed it as a BPMN 2.0 web modeler into your applications and customize it to suit your needs.
Learn more
Try Online



ERP/1: KV Storage SYNRC KVS

https://n2o.dev/books/n2o.pdf

CEPH or raw rocksdb NVMe SSD

https://5ht.co/snia-kv-api-1.1.pdf

https://kvs.n2o.dev/

Samsung Intel

https://erp.uno/storage/

SYNRC KVS

SNIA KV API 2020

ERP/1: KV Storage

KVS/NVMe Practical Model (System F/MLTT)

```
axiom get : Π (f k v: U), f -> k -> IO (Maybe v)
axiom put : Π (r: U), r -> IO StoreResult
axiom delete : Π (f k: U), f -> k -> StoreResult
axiom index : Π (f p v r: U), f -> Atom -> v -> List r

axiom next : Reader -> IO Reader
axiom prev : Reader -> IO Reader
axiom take : Reader -> IO Reader
axiom drop : Reader -> IO Reader
axiom save : Reader -> IO Reader
axiom append : Π (f r: U), f -> r -> IO StoreResult
axiom remove : Π (f r: U), f -> r -> IO StoreResult
```

KVS Objects (Schema)

#writer
#reader
#id_seq
#it
#ite
#iter
#kvs

#schema
#table

rocksdb

```
1> {ok,Ref} = rocksdb:open("hey",[{create_if_missing,true}]).
2> rocksdb:put(Ref, <<"/users/1">>,<<"maxim">>,[{sync,true}]).
3> rocksdb:put(Ref, <<"/users/2">>,<<"evhen">>,[{sync,true}]).
4> rocksdb:put(Ref, <<"/users/3">>,<<"yuriy">>,[{sync,true}]).
5> rocksdb:put(Ref, <<"/staff/1">>,<<"maxim">>,[{sync,true}]).
6> rocksdb:put(Ref, <<"/staff/2">>,<<"evhen">>,[{sync,true}]).
7> rocksdb:put(Ref, <<"/staff/3">>,<<"yuriy">>,[{sync,true}]).
8> {ok,I} = rocksdb:iterator(Ref,[]).
9> rocksdb:iterator_move(I,{seek,<<"/staff/">>}).
10> rocksdb:iterator_move(I,next).
11> rocksdb:iterator_move(I,next).
12> rocksdb:iterator_move(I,next).
13> rocksdb:iterator_move(I,{seek,<<"/users/">>}).
14> rocksdb:iterator_move(I,next).
15> rocksdb:iterator_move(I,next).
16> rocksdb:iterator_move(I,next).
```

KVS

```
1> kvs:ver().
{version,"KVS ROCKSDB"}
2> rr(kvs).
[emails,id_seq,it,iter,kvs,reader,schema,table,writer]
3> kvs:join().
ok
4> kvs:put(#emails{id=1,email="maxim"}).
5> kvs:put(#emails{id=2,email="eugen"}).
6> kvs:put(#writer{id=2}).
7> kvs:put(#writer{id=1}).
8> kvs:all(writer).
[#writer{id = 1,count = 0,cache = [],args = [],first = []},
 #writer{id = 2,count = 0,cache = [],args = [],first = []}]
9> kvs:all(emails).
[#emails{id = 1,next = [],prev = [],email = "maxim"},
 #emails{id = 2,next = [],prev = [],email = "eugen"}]
10> kvs:add(#writer{id=chain,args=#emails{email="maxim@synrc.com"}}).
11> kvs:add(#writer{id=chain,args=#emails{email="yuriy@synrc.com"}}).
12> kvs:add(#writer{id=chain,args=#emails{email="eugen@synrc.com"}}).
13> kvs:all(chain).
[#emails{id = 1555244691729330000,next = [],prev = [], email = "maxim@synrc.com"},
 #emails{id = 1555244699905648000,next = [],prev = [], email = "eugen@synrc.com"},
 #emails{id = 1555244696660271000,next = [],prev = [], email = "yuriy@synrc.com"}]
```

KVS Links

https://tonpa.guru/stream/2019/2019-09-26%20BPE%20SCHED.htm
https://tonpa.guru/stream/2019/2019-04-13%20Новая%20версия%20KVS.htm
https://tonpa.guru/stream/2018/2018-11-13%20Новая%20версия%20KVS.htm
https://tonpa.guru/stream/2017/2017-04-19%20N2O%20over%20MQTT%20+%20KVS.txt
https://tonpa.guru/stream/2017/2017-03-01%20KVS%20—%20%20DSL%20для%20Алкотрейдинга!.txt
https://tonpa.guru/stream/2016/2016-03-29%20KVS%20intro.txt
https://tonpa.guru/stream/2016/2016-09-24%20KVS%20STREAMS.txt
https://tonpa.guru/stream/2014/2014-09-29%20Версионирование%20схем%20в%20KVS.txt

Zen Crypted

Embeddings

```
-record(it, { id = [] :: [] | integer() } ).
```

kvs_st

Zen Crypted

Lists

```
-record(ite, { id    = [] :: [] | integer(),
               next  = [] :: [] | integer() } ).
```

kvs_stre

Zen Crypted

Trees

```
-record(iter, { id    = [] :: [] | integer(),
                next  = [] :: [] | integer(),
                prev  = [] :: [] | integer() } ).
```

kvs_stream

Zen Crypted

Groupoid Infinity Cafe

Learning Materials Since 2016

https://cafe.groupoid.space/

Zen Crypted